NewsChecker

Application Décentralisée de Vérification des Informations

Contrat Intelligent **SmartPy** sur **Tezos Ghostnet** · Interface **React 19** + **Taquito 24**

Réalisé par

Maram **Arfaoui**

Abderrazzak **El Bourkadi**

Ayoub **Et-Toubi**

Asmaa **Assaid**

Mohamed **Cheikh**

Réseau : Tezos Ghostnet (Testnet)

Contrat : `KT1H49BAbxYfBReehN4dShGpm6jVWapXPgeq`

Version : 1.0.0 · Mars 2026

Résumé

NewsChecker est une application décentralisée (DApp) déployée sur la blockchain **Tezos** qui permet la vérification collective et transparente d'articles de presse. Le système s'appuie sur un contrat intelligent écrit en **SmartPy**, combinant un mécanisme de mise en jeu (*staking*) économique, un système de réputation progressif, et un vote pondéré résistant aux attaques Sybil.

Les membres s'inscrivent en verrouillant une mise minimale en Tez, soumettent des articles à vérifier, et votent sur leur authenticité. Le poids de chaque vote est nul jusqu'à ce qu'un membre atteigne le seuil de réputation (5 points, soit le statut *Voter Confirmé*), puis devient égal à sa réputation. La finalisation d'un vote suit deux modes : **Mode A** (quorum atteint + délai minimum écoulé) ou **Mode B** (deadline dépassée avec au moins un vote). Les votants corrects voient leur réputation augmenter de $+2$, les votants incorrects subissent une pénalité de -3.

L'interface web **React**, connectée via la bibliothèque **Taquito** et le protocole **Beacon**, offre quatre panneaux fonctionnels : Dashboard, News, Member et Admin. Un mode Démo permet l'exploration complète sans portefeuille réel.

Table des matières

1 Introduction **4**
- 1.1 Problématique . . . 4
- 1.2 Objectifs du projet . . . 4
- 1.3 Contributions principales . . . 4

2 Architecture du Système **5**
- 2.1 Vue d'ensemble technologique . . . 5
- 2.2 Stack technique . . . 6

3 Smart Contract NewsChecker **6**
- 3.1 Vue d'ensemble . . . 6
- 3.2 Structures de données . . . 7
  - 3.2.1 Storage du contrat . . . 7
  - 3.2.2 Enregistrements SmartPy . . . 8
- 3.3 Les 9 points d'entrée . . . 8
  - 3.3.1 `register` — Inscription . . . 8
  - 3.3.2 `vote` — Vote pondéré par réputation . . . 8
  - 3.3.3 `finalize` — Clôture du vote . . . 9
  - 3.3.4 `add_stake`, `withdraw_stake` et `update_params` . . . 9
  - 3.3.5 `submit_news` — Soumission avec dépôt anti-spam . . . 10
  - 3.3.6 `finalize` — Clôture sans boucle (pull pattern) . . . 10
  - 3.3.7 `claim_reward` — Récompense individuelle (pull pattern) . . . 11
  - 3.3.8 `cancel_news` — Annulation par l'auteur . . . 11
- 3.4 Système de réputation . . . 12
  - 3.4.1 Machine à états . . . 12
  - 3.4.2 Évolution de la réputation . . . 12
- 3.5 Protection anti-Sybil . . . 12

4 Interface DApp (React) **13**
- 4.1 Architecture des composants . . . 13
- 4.2 WalletContext . . . 14

5 Interface Utilisateur — Captures d'écran **14**
- 5.1 Page d'accueil (non connecté) . . . 15
- 5.2 Dashboard . . . 15
- 5.3 Panneau News . . . 16
- 5.4 Panneau Membre . . . 16
- 5.5 Panneau Admin . . . 17

6 Scénarios de Test **17**
6.1 Suite de tests SmartPy 17
6.2 Codes d'erreur 19

7 Analyse de Sécurité **20**
7.1 Vecteurs d'attaque et défenses 20
7.2 Progression de réputation 21

8 Guide de Déploiement **21**
8.1 Déploiement du contrat sur Ghostnet 21
8.2 Lancement du frontend 22

9 Conclusion **22**
9.1 Perspectives d'amélioration 23

1 Introduction

1.1 Problématique

La prolifération de fausses informations sur internet constitue l'un des défis majeurs de notre époque numérique. Les plateformes centralisées de fact-checking présentent des vulnérabilités inhérentes : biais éditoriaux, point de défaillance unique, manque de transparence dans les processus de décision, et risque de censure arbitraire.

Problèmes des systèmes centralisés

- **Point de confiance unique :** L'autorité centrale peut être corrompue ou compromise
- **Attaques Sybil :** Un acteur malveillant peut créer de multiples comptes pour manipuler les votes
- **Absence de traçabilité :** Les décisions de modération sont opaques et non vérifiables
- **Résistance à la censure :** Aucune garantie contre la suppression d'informations légitimes

1.2 Objectifs du projet

NewsChecker propose une alternative décentralisée reposant sur les propriétés fondamentales de la blockchain **Tezos** :

Objectifs principaux

1. **Décentralisation :** Aucune autorité centrale — le contrat intelligent est la loi
2. **Transparence :** Toutes les données (votes, réputations, stakes) sont publiques sur la chaîne
3. **Résistance Sybil :** Le mécanisme de réputation progressive neutralise les comptes fantômes
4. **Incitation économique :** Le staking engage financièrement les participants à voter honnêtement
5. **Gouvernance évolutive :** L'administrateur peut ajuster les paramètres sans redéploiement

1.3 Contributions principales

- Conception d'un contrat intelligent **SmartPy** avec 9 points d'entrée (*entry-points*)

- Implémentation d'un système de réputation bi-état (Probatoire / Confirmé)
- Double mécanisme de finalisation pour gérer les cas de faible participation
- Mécanisme **pull-over-push** : distribution des récompenses via `claim_reward` (pas de boucles)
- Dépôt anti-spam sur soumission : perdu si FAKE, rendu si RÉEL
- Verrouillage du stake pendant les votes actifs (`active_votes`)
- Interface DApp **React** complète avec mode Démo intégré
- Suite de tests automatisés (11 scénarios couvrant les cas nominaux et adversariaux)

2 Architecture du Système

2.1 Vue d'ensemble technologique

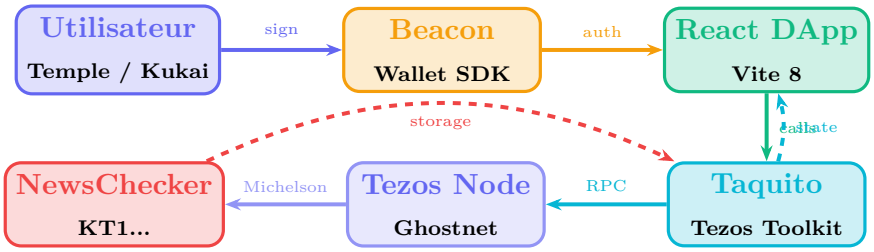


Fig. 1 : *Architecture globale du système NewsChecker*

2.2 Stack technique

Couche	Technologie	Rôle
Smart Contract	SmartPy 0.20	Logique métier on-chain (Python → Michelson)
Réseau	Tezos Ghostnet	Blockchain Layer-1, consensus proof-of-stake
RPC Client	Taquito 20	Communication JavaScript ↔ nœud Tezos
Wallet	Beacon SDK 4	Protocole de connexion portefeuille universel
Frontend	React 19 + Vite 8	Interface utilisateur réactive
Styles	Tailwind CSS 4	Design system utilitaire
Icônes	Lucide React	Bibliothèque d'icônes SVG
Notifications	React Hot Toast	Retours utilisateur en temps réel

Tab. 1 : *Stack technique complet du projet*

3 Smart Contract NewsChecker

3.1 Vue d'ensemble

Le contrat `NewsChecker` est écrit en **SmartPy** et orchestre l'intégralité de la logique de vérification des nouvelles. Il gère les membres, les articles, les votes, et les réputations de manière entièrement autonome et transparente.

Paramètres d'initialisation

Paramètre	Valeur démo	Description
`admin`	`tz1Admin...`	Adresse administrateur
`min_stake`	1 000 000 µtez	Mise minimale (1 tez)
`quorum`	3	Votes requis (Mode A)
`rep_threshold`	5	Seuil réputation pondérée
`vote_delay`	3 600 s	Délai min. avant finalisation (1h)
`vote_duration`	86 400 s	Durée maximale du vote (24h)
`news_deposit`	500 000 µtez	Dépôt anti-spam par soumission (0,5 tez)

3.2 Structures de données

3.2.1 Storage du contrat

Champ	Type SmartPy	Description
`admin`	`sp.address`	Adresse administrateur
`min_stake`	`sp.mutez`	Mise minimale requise
`quorum`	`sp.nat`	Nb. votes pour Mode A
`rep_threshold`	`sp.nat`	Seuil réputation pondérée
`vote_delay`	`sp.int`	Délai min (secondes)
`vote_duration`	`sp.int`	Durée max (secondes)
`news_deposit`	`sp.mutez`	Dépôt anti-spam par soumission
`next_news_id`	`sp.nat`	Compteur auto-incrémenté
`members`	`big_map[addr, Member]`	Membres inscrits
`news`	`big_map[nat, News]`	Articles soumis
`votes`	`big_map[pair(nat,addr),bool]`	Votes (bigmap plat)
`claimed`	`big_map[pair(nat,addr),bool]`	Récompenses réclamées

Tab. 2 : *Structure du storage du contrat NewsChecker*

3.2.2 Enregistrements SmartPy

```
# Enregistrement d'un membre
sp.record(
    reputation   = sp.nat,     # Points de reputation (debut: 1)
    staked       = sp.mutez,   # Montant verrouille en Tez
    active       = sp.bool,    # Statut actif/inactif
    active_votes = sp.nat      # Nb. votes actifs (verrou stake)
)

# Enregistrement d'un article
sp.record(
    author      = sp.address,     # Soumetteur
    title       = sp.string,      # Titre
    url         = sp.string,      # URL source
    status      = sp.nat,         # 1=Vote, 2=Reel, 3=Fake, 4=Annulee
    votes_real  = sp.nat,         # Somme ponderee REEL
    votes_fake  = sp.nat,         # Somme ponderee FAKE
    count_real  = sp.nat,         # Votes bruts REEL
    count_fake  = sp.nat,         # Votes bruts FAKE
    submit_time = sp.timestamp,   # Horodatage soumission
    deposit     = sp.mutez        # Depot anti-spam de l'auteur
)
```

Listing 1: *Structure Member et NewsItem*

3.3 Les 9 points d'entrée

3.3.1 `register` — Inscription

```
@sp.entrypoint
def register(self):
    assert not self.data.members.contains(sp.sender), "ALREADY_REGISTERED"
    assert sp.amount >= self.data.min_stake, "INSUFFICIENT_STAKE"
    self.data.members[sp.sender] = sp.record(
        reputation   = sp.nat(1),    # Mode probatoire (poids=0)
        staked       = sp.amount,
        active       = True,
        active_votes = sp.nat(0)     # Aucun vote actif au depart
    )
```

Listing 2: *Entrypoint register*

Logique register

L'inscription requiert un paiement on-chain d'au moins `min_stake` mutez. La réputation initiale de 1 place le membre en mode **Probatoire** — ses votes ne comptent pas dans le total pondéré tant qu'il n'atteint pas le seuil de 5.

3.3.2 `vote` — Vote pondéré par réputation

```
@sp.entrypoint
def vote(self, news_id, opinion):
    n = self.data.news[news_id]
    assert n.status == sp.nat(1), "NOT_IN_VOTING"
    deadline = sp.add_seconds(n.submit_time, self.data.vote_duration)
    assert sp.now < deadline, "VOTING_ENDED"      # Deadline
    m = self.data.members[sp.sender]
```

```
    assert m.staked >= self.data.min_stake, "STAKE_TOO_LOW"
    key = (news_id, sp.sender)
    assert not self.data.votes.contains(key), "ALREADY_VOTED"
    # Poids: 0 si probatoire, reputation si confirme
    w = sp.nat(0)
    if m.reputation >= self.data.rep_threshold:
        w = m.reputation
    # Enregistrement avec poids
    if opinion:
        # votes_real += w, count_real += 1
    else:
        # votes_fake += w, count_fake += 1
```

Listing 3: *Entrypoint vote avec poids de reputation*

3.3.3 `finalize` — Clôture du vote

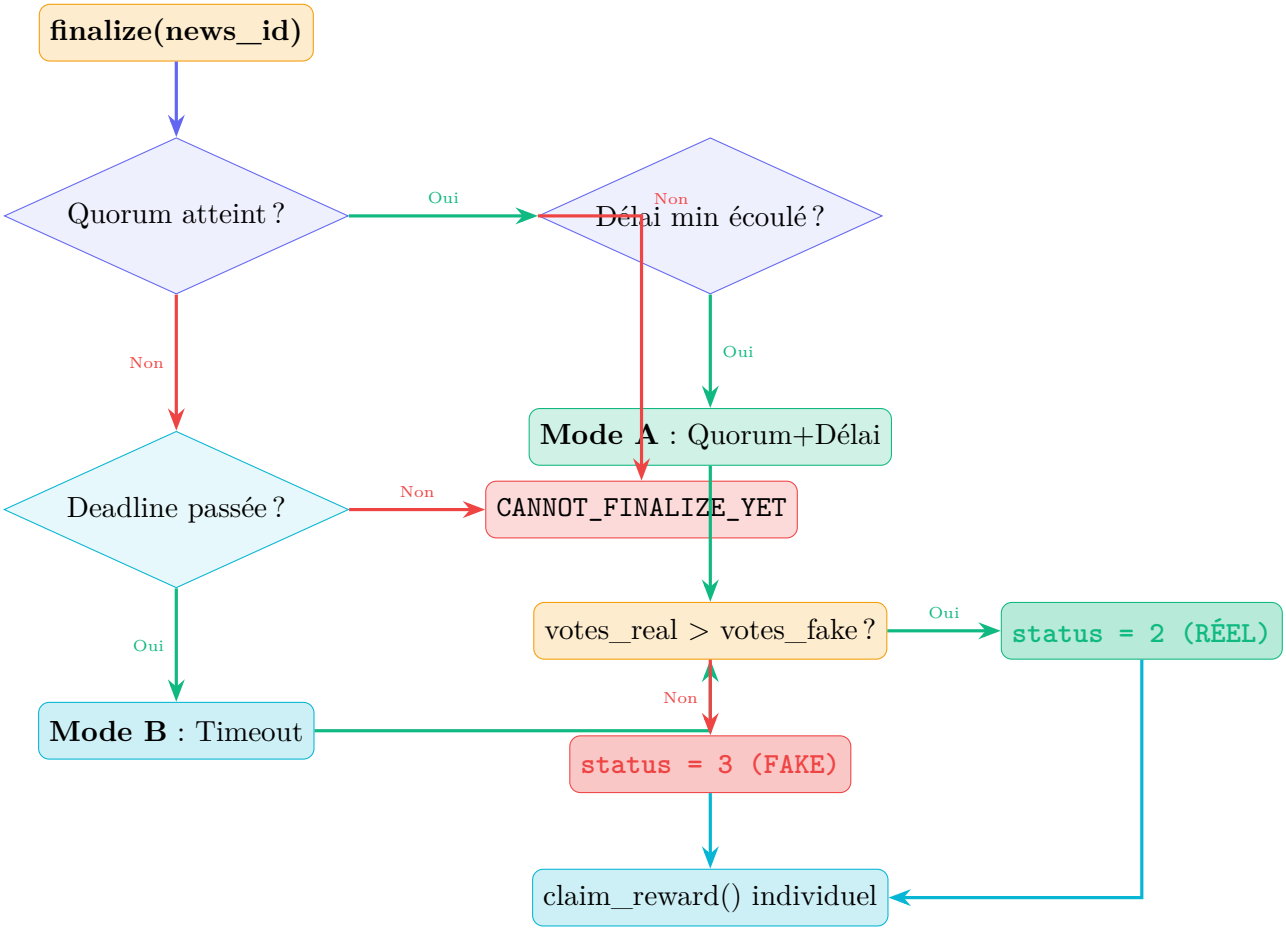


Fig. 2 : *Flowchart du processus de finalisation (deux modes)*

3.3.4 `add_stake`, `withdraw_stake` et `update_params`

```
@sp.entrypoint
def add_stake(self):
    assert self.data.members.contains(sp.sender), "NOT_REGISTERED"
    assert sp.amount > sp.mutez(0), "ZERO_AMOUNT"
    self.data.members[sp.sender].staked += sp.amount  # Mise a jour directe

```

```
@sp.entrypoint
def withdraw_stake(self, amount):
    m = self.data.members[sp.sender]
    assert m.active_votes == sp.nat(0), "ACTIVE_VOTES"  # Verrou vote
    assert m.staked >= amount, "NOT_ENOUGH"             # Retrait total OK
    self.data.members[sp.sender].staked -= amount
    sp.send(sp.sender, amount)  # Transfert on-chain

@sp.entrypoint
def update_params(self, new_min_stake, new_quorum, new_rep_threshold,
                  new_vote_delay, new_vote_duration, new_news_deposit):
    assert sp.sender == self.data.admin, "NOT_ADMIN"
    self.data.min_stake     = new_min_stake
    self.data.quorum        = new_quorum
    self.data.rep_threshold = new_rep_threshold
    self.data.vote_delay    = new_vote_delay
    self.data.vote_duration = new_vote_duration
    self.data.news_deposit  = new_news_deposit  # Nouveau parametre
```

Listing 4: *Gestion du stake et paramètres admin*

3.3.5 `submit_news` — Soumission avec dépôt anti-spam

```
@sp.entrypoint
def submit_news(self, title, url):
    assert self.data.members.contains(sp.sender), "NOT_REGISTERED"
    assert sp.amount >= self.data.news_deposit, "INSUFFICIENT_DEPOSIT"
    nid = self.data.next_news_id
    self.data.news[nid] = sp.record(
        author=sp.sender, title=title, url=url,
        status=sp.nat(1),  # EN_VOTE
        ...,
        deposit=sp.amount  # Sauvegarde du depot
    )
    self.data.next_news_id += sp.nat(1)
```

Listing 5: *Entrypoint submit_news avec dépôt*

Mécanisme anti-spam du dépôt

Le dépôt est prélevé à la soumission et géré comme suit :

- **Si RÉEL** : dépôt remboursé à l'auteur dans `finalize`
- **Si FAKE** : dépôt conservé dans le contrat (pénalité)
- **Si ANNULÉE** : dépôt remboursé via `cancel_news` (0 votes, deadline passée)

3.3.6 `finalize` — Clôture sans boucle (pull pattern)

```
@sp.entrypoint
def finalize(self, news_id):
    n = self.data.news[news_id]
    assert n.status == sp.nat(1), "NOT_IN_VOTING"
    # ... verifications quorum / delai / deadline
    # Determination du resultat
    final_status = sp.nat(3)  # FAKE par defaut
    if n.votes_real > n.votes_fake:
        final_status = sp.nat(2)  # REAL
```

```
    self.data.news[news_id].status = final_status
    # Depot rendu UNIQUEMENT si REAL (anti-spam)
    if final_status == sp.nat(2):
        sp.send(n.author, n.deposit)
    # Pas de boucle sur les votants - pattern pull
```

Listing 6: *Entrypoint finalize — pas de boucle de distribution*

3.3.7 `claim_reward` — Récompense individuelle (pull pattern)

```
@sp.entrypoint
def claim_reward(self, news_id):
    n   = self.data.news[news_id]
    assert n.status == sp.nat(2) or n.status == sp.nat(3), "NOT_FINALIZED"
    key = sp.pair(news_id, sp.sender)
    assert self.data.votes.contains(key),       "NOT_A_VOTER"
    assert not self.data.claimed.contains(key), "ALREADY_CLAIMED"
    self.data.claimed[key] = True
    # Liberer le verrou de stake [P13]
    self.data.members[sp.sender].active_votes -= sp.nat(1)
    # Mise a jour reputation
    voter_opinion = self.data.votes[key]
    is_real = (n.status == sp.nat(2))
    if voter_opinion == is_real:
        self.data.members[sp.sender].reputation += sp.nat(2)  # Bon vote
    else:
        # Mauvais vote : -3, plancher = 1
        if self.data.members[sp.sender].reputation > sp.nat(4):
            self.data.members[sp.sender].reputation -= sp.nat(3)
        else:
            self.data.members[sp.sender].reputation = sp.nat(1)
```

Listing 7: *Entrypoint claim_reward — chaque votant réclame individuellement*

Avantages du pattern pull-over-push

Le pattern pull remplace la boucle de distribution présente dans la v1 :

- **Pas de limite de gas** : chaque `claim_reward` est une transaction indépendante
- **Sécurité** : impossible de bloquer le contrat en ayant trop de votants
- **Flexibilité** : chaque votant réclame quand il le souhaite

3.3.8 `cancel_news` — Annulation par l'auteur

```
@sp.entrypoint
def cancel_news(self, news_id):
    n = self.data.news[news_id]
    assert n.status == sp.nat(1),         "NOT_IN_VOTING"
    assert sp.sender == n.author,         "NOT_AUTHOR"
    assert n.count_real + n.count_fake == sp.nat(0), "HAS_VOTES"
    deadline = sp.add_seconds(n.submit_time, self.data.vote_duration)
    assert sp.now >= deadline,            "DEADLINE_NOT_PASSED"
    self.data.news[news_id].status = sp.nat(4)  # ANNULEE
    sp.send(n.author, n.deposit)  # Depot rendu (news ignoree, pas du spam)
```

Listing 8: *Entrypoint cancel_news*

3.4 Système de réputation

3.4.1 Machine à états

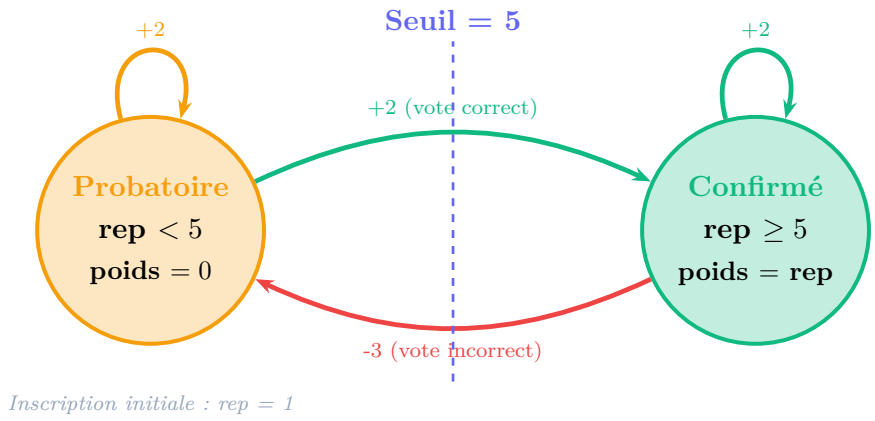


Fig. 3 : *Machine à états du système de réputation*

3.4.2 Évolution de la réputation

Situation	Rép. avant	Delta	Rép. après
Vote correct (probatoire, tour 1)	1	+2	3
Vote correct (probatoire, tour 2)	3	+2	**5 Confirmé**
Vote correct (confirmé)	5	+2	7
Vote incorrect (confirmé)	7	-3	4 ← Probatoire
Vote incorrect (rep ≤ 4)	4	-3 (min 1)	1

Tab. 3 : *Évolution de la réputation selon les votes*

3.5 Protection anti-Sybil

Résistance aux attaques Sybil

Un attaquant créant N comptes en probatoire (rep = 1) ne peut pas influencer le résultat car le **poids pondéré** de chacun est 0. Seuls les membres **Confirmés** (rep ≥ 5) ont un poids effectif.

Exemple (test Sybil) : 5 attaquants (poids = 0) votent RÉEL vs 2 confirmés

(poids = 5 chacun) votant FAKE :

$$\text{votes_real} = 5 \times 0 = 0 \quad \text{vs} \quad \text{votes_fake} = 2 \times 5 = 10$$

$$\Rightarrow \text{Résultat : } \textbf{FAKE} \quad \checkmark$$

4 Interface DApp (React)

4.1 Architecture des composants

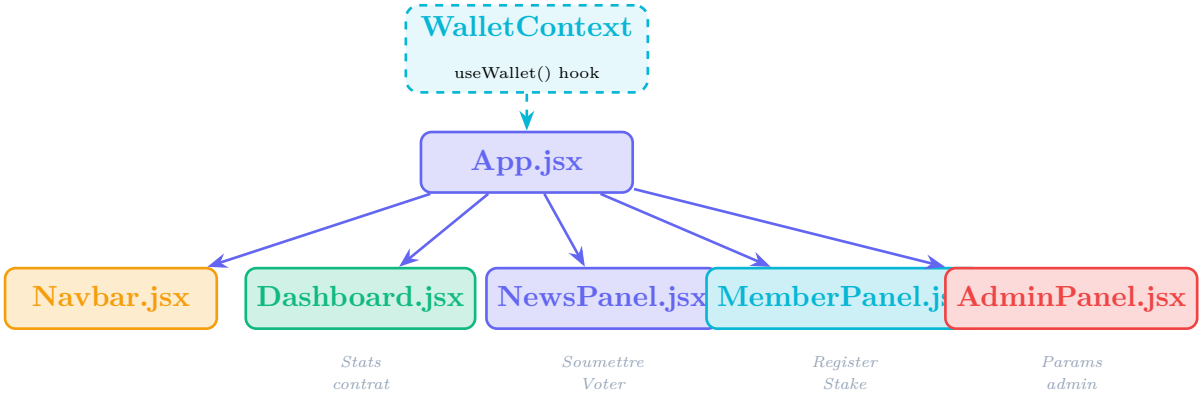


Fig. 4 : *Arbre des composants React de l'application*

4.2 WalletContext

Propriété/Méthoc	Type	Description
`wallet`	`BeaconWallet`	Instance du wallet Tezos
`address`	`string|null`	Adresse connectée
`storage`	`object|null`	Storage du contrat
`isDemo`	`boolean`	Mode démo actif
`connect()`	`async function`	Connecte le wallet Beacon
`connectDemo()`	`async function`	Active le mode démo
`disconnect()`	`function`	Déconnecte l'utilisateur
`fetchStorage()`	`async function`	Recharge le storage
`callContract()`	`async function`	Appelle un entrypoint

Tab. 4 : *API exposée par WalletContext via useWallet()*

```
const tezos = new TezosToolkit('https://ghostnet.tezos.marigold.dev');

const beaconWallet = new BeaconWallet({
  name: 'NewsChecker DApp',
  network: { type: NetworkType.GHOSTNET }  // Reseau dans le constructeur
});

tezos.setWalletProvider(beaconWallet);

// Appel d'un entrypoint
const contract = await tezos.wallet.at(CONTRACT_ADDRESS);
await contract.methods.vote(newsId, opinion).send();
```

Listing 9: *Initialisation TezosToolkit et BeaconWallet*

5 Interface Utilisateur — Captures d'écran

5.1 Page d'accueil (non connecté)

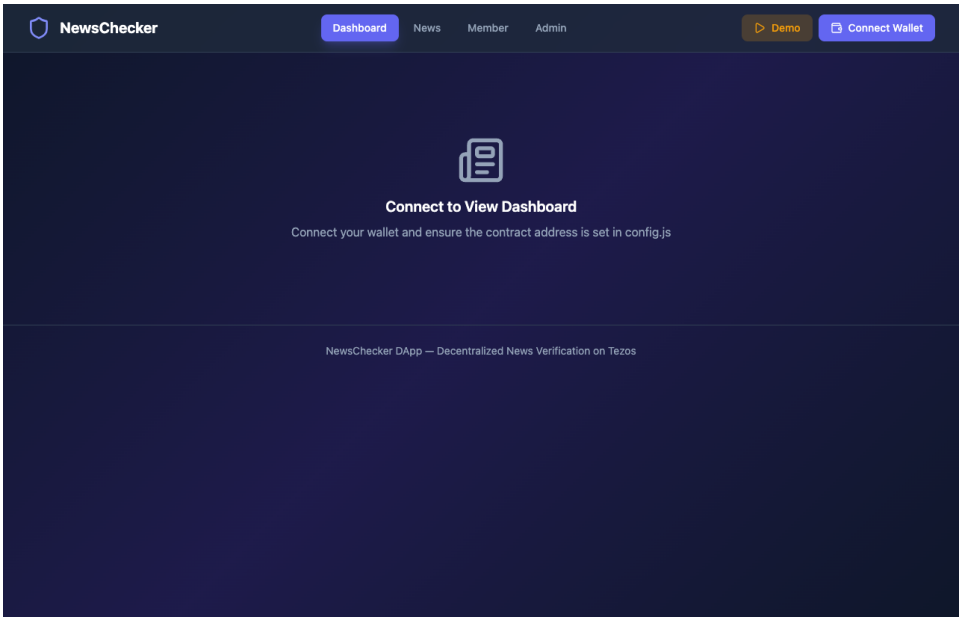
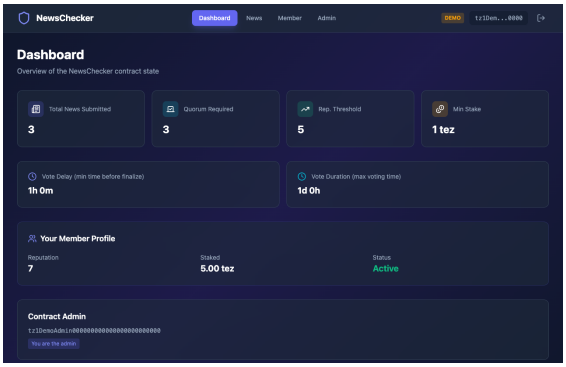
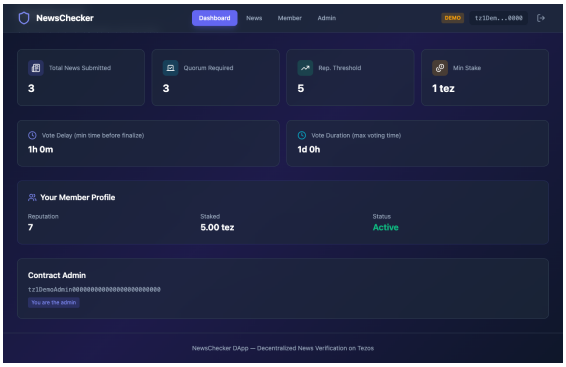


Fig. 5 : *Page d'accueil — Avant connexion du portefeuille. Les boutons* Demo *et* Connect Wallet *sont visibles dans la barre de navigation.*

5.2 Dashboard



(a) *Statistiques : Total News=3, Quorum=3, Rep. Threshold=5, Min Stake=1 tez*



(b) *Délais, profil membre (rep=7, stake=5 tez, Active), adresse admin*

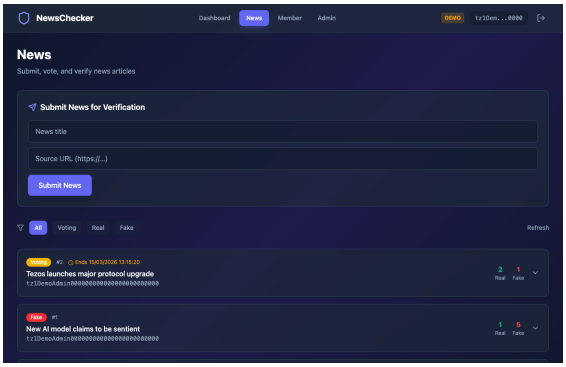
Fig. 6 : *Dashboard — Vue d'ensemble de l'état du contrat (mode démo)*

Métriques affichées sur le Dashboard

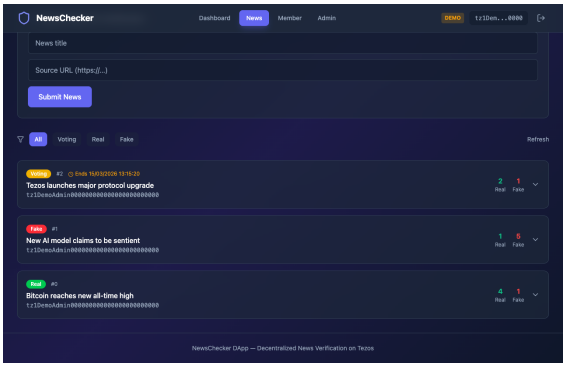
- **Total News Submitted :** 3 articles en mode démo (Voting, Fake, Real)
- **Quorum Required :** 3 votes minimum pour déclencher le Mode A
- **Rep. Threshold :** 5 — seuil pour devenir Voter Confirmé
- **Min Stake :** 1 tez — mise économique minimale
- **Vote Delay :** 1h 0m — délai avant finalisation Mode A

- **Vote Duration :** 1j 0h — durée maximale du vote

5.3 Panneau News



(a) *Formulaire de soumission d'un article (titre + URL)*



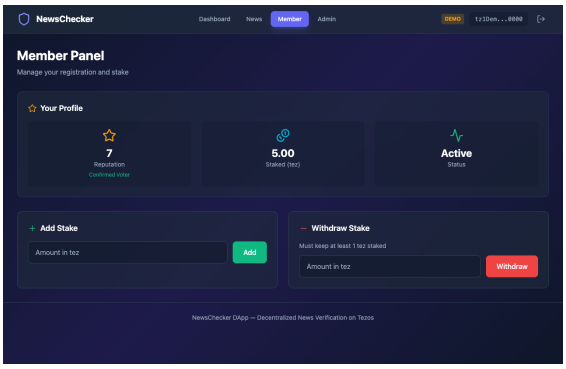
(b) *Liste des articles avec badges de statut colorés*

Fig. 7 : *Panneau News — Soumission et consultation des articles*

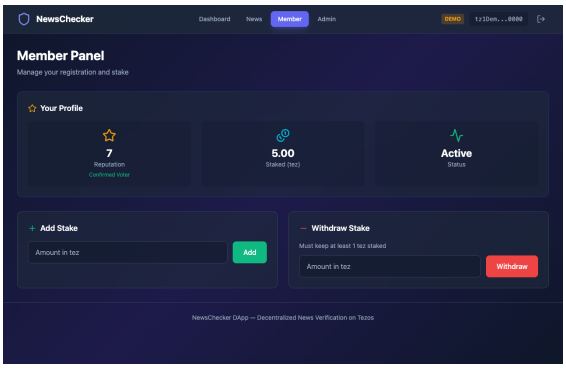
Les trois articles démo illustrent les statuts possibles :

- `#2 Voting` — "Tezos launches major protocol upgrade" (2 REAL / 1 FAKE)
- `#1 Fake` — "New AI model claims to be sentient" (1 REAL / 5 FAKE)
- `#0 Real` — "Bitcoin reaches new all-time high" (4 REAL / 1 FAKE)

5.4 Panneau Membre



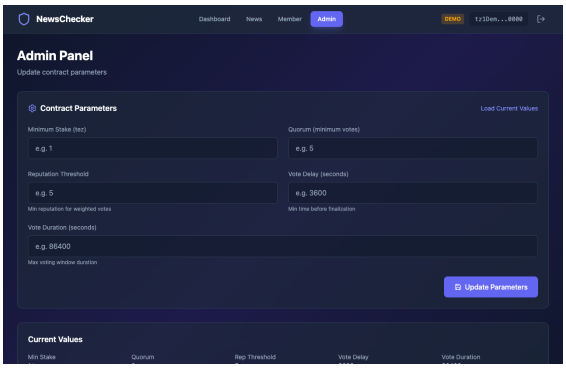
(a) *Profil : Réputation 7 (Confirmé), Stake 5.00 tez, Status Active*



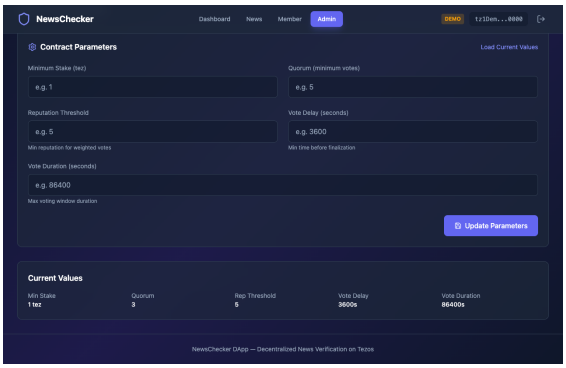
(b) *Formulaires Add Stake et Withdraw Stake*

Fig. 8 : *Panneau Membre — Gestion de l'inscription et du stake*

5.5 Panneau Admin



(a) *Paramètres du contrat : Min Stake, Quorum, Rep Threshold*



(b) *Vote Delay, Vote Duration, bouton Update Parameters*

Fig. 9 : *Panneau Admin — Mise à jour des paramètres (admin uniquement)*

6 Scénarios de Test

6.1 Suite de tests SmartPy

Nom du test	Mode testé	Résultat attendu
`test_register`	Inscription	Membre créé avec rep=1, active_votes=0 ✓
`test_staking`	Stake add/withdraw	Retrait total autorisé, solde vérifié ✓
`test_stake_locked_during_vote`	Verrou stake [P13]	`ACTIVE_VOTES` pendant vote actif ✓
`test_submit_news_depo`	Dépôt anti-spam [P7]	`INSUFFICIENT_DEPOSIT` si manquant ✓
`test_vote`	Vote pondéré	Poids=0 en probatoire, VOTING_ENDED après deadline ✓
`test_finalize`	Mode A + Mode B	Finalise par quorum+délai ou deadline ✓
`test_deposit_anti_spam`	Anti-spam réel	Dépôt perdu si FAKE, rendu si RÉEL ✓
`test_claim_reward`	Pull pattern [P8-10]	rep+2 (bon vote), rep plancher=1 (mauvais) ✓

Nom du test	Mode testé	Résultat attendu
`test_cancel_news`	Annulation auteur	Status=4, dépôt rendu si 0 votes + deadline ✓
`test_reputation_full`	Cycle complet rep.	rep 1→3→5→7 sur 3 tours, votes pondérés ✓
`test_sybil_attack`	Résistance Sybil	3 Sybil (poids=0) vs 2 confirmés (rep=5) → FAKE ✓
`test_admin`	Admin params	6 paramètres mis à jour, non-admin bloqué ✓

Tab. 5 : *Suite complète des 11 tests SmartPy*

6.2 Codes d'erreur

Condition	Code d'erreur	Entrypoint
Déjà inscrit	ALREADY_REGISTERED	register
Mise insuffisante	INSUFFICIENT_STAKE	register
Non inscrit	NOT_REGISTERED	Tous
Dépôt insuffisant	INSUFFICIENT_DEPOSIT	submit_news
Titre vide	EMPTY_TITLE	submit_news
URL vide	EMPTY_URL	submit_news
News inexistante	NEWS_NOT_FOUND	vote, finalize, claim_reward
Statut incorrect	NOT_IN_VOTING	vote, finalize, cancel_news
Deadline dépassée	VOTING_ENDED	vote
Stake trop bas	STAKE_TOO_LOW	vote
Déjà voté	ALREADY_VOTED	vote
Finalisation impossible	CANNOT_FINALIZE_YET	finalize
Aucun vote	NO_VOTES	finalize
Non finalisée	NOT_FINALIZED	claim_reward
Pas votant	NOT_A_VOTER	claim_reward
Déjà réclamé	ALREADY_CLAIMED	claim_reward
Solde insuffisant	NOT_ENOUGH	withdraw_stake
Votes actifs présents	ACTIVE_VOTES	withdraw_stake
Pas l'auteur	NOT_AUTHOR	cancel_news
Votes déjà reçus	HAS_VOTES	cancel_news
Deadline pas passée	DEADLINE_NOT_PASSED	cancel_news
Non administrateur	NOT_ADMIN	update_params

Tab. 6 : *Codes d'erreur du contrat (22 conditions)*

7 Analyse de Sécurité

7.1 Vecteurs d'attaque et défenses

Attaque	Mécanisme de défense	Efficacité
Sybil (multi-comptes)	Poids=0 en probatoire jusqu'à rep≥5	Très élevée
Vote précipité	Mode A : délai min obligatoire	Élevée
Collusion (inactifs)	Mode B : finalisation après deadline	Élevée
Manipulation admin	Admin limité à update_params uniquement	Bonne
Vote sans enjeu	Staking minimum obligatoire	Élevée
Double vote	Clé `pair(news_id, address)` unique (bigmap plat)	Totale
Spam soumissions	Dépôt anti-spam perdu si FAKE	Élevée
Retrait stake pendant vote	Compteur `active_votes` verrouille le stake	Totale
Boucle gas exhaustion	Pattern pull : `claim_reward` individuel, pas de boucle	Totale

Tab. 7 : *Analyse des vecteurs d'attaque et contre-mesures*

7.2 Progression de réputation

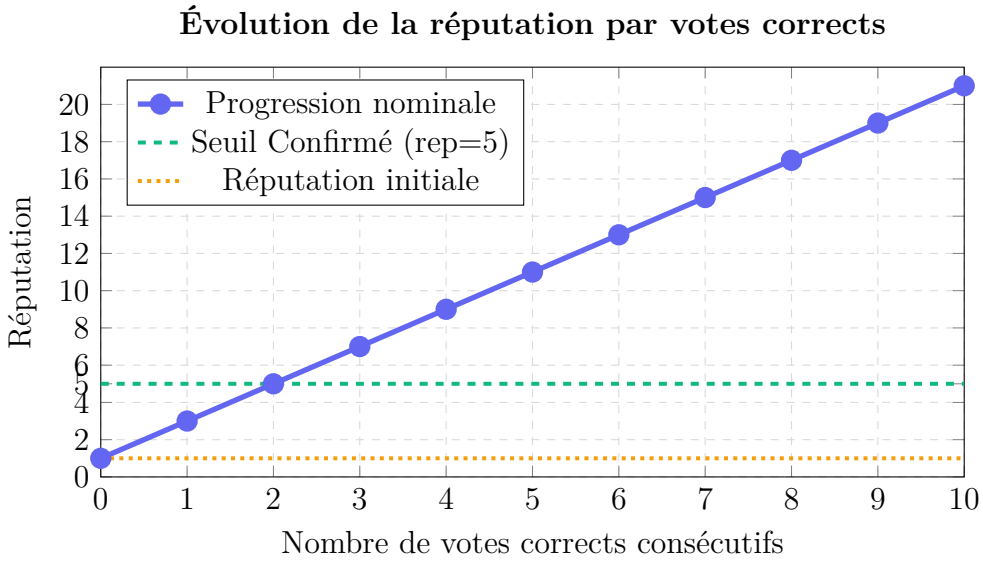


Fig. 10 : *Progression de la réputation sur 10 votes corrects consécutifs*

8 Guide de Déploiement

8.1 Déploiement du contrat sur Ghostnet

1. **Obtenir des tez de test** via le faucet Ghostnet : https://faucet.ghostnet.teztnets.com/
2. **Compiler et déployer** via SmartPy IDE : https://smartpy.io/ide
   - Coller le code du contrat dans l'éditeur
   - Cliquer **Run** puis naviguer vers **Deploy**
   - Sélectionner le réseau **Ghostnet**
   - Connecter Temple ou Kukai Wallet
   - Confirmer la transaction (coût : environ 1 tez)
3. **Récupérer l'adresse** `KT1...` du contrat déployé via https://ghostnet.tzkt.io
4. **Configurer le frontend** dans `src/config.js` :

```
export const CONTRACT_ADDRESS = 'KT1VotreAdresseContratIci...';
export const RPC_URL          = 'https://ghostnet.tezos.marigold.dev';
export const NETWORK          = 'ghostnet';
```

Listing 10: *Configuration src/config.js*

8.2 Lancement du frontend

```
# Installer les dependances
npm install

# Lancer en developpement (http://localhost:5173)
npm run dev

# Build de production
npm run build

# Preview du build de production
npm run preview
```

Listing 11: *Commandes de démarrage*

Wallet requis pour le mode réel

Pour interagir avec le contrat déployé, les utilisateurs ont besoin d'un portefeuille Tezos compatible Beacon :

- **Temple Wallet :** https://templewallet.com (recommandé pour Ghostnet)
- **Kukai :** https://kukai.app

En attendant le déploiement, le **Mode Démo** permet de tester toutes les fonctionnalités sans portefeuille.

9 Conclusion

NewsChecker démontre la faisabilité d'un système de vérification des nouvelles décentralisé, transparent et résistant aux manipulations. Les principaux apports du projet sont :

Résultats obtenus

- **Contrat fonctionnel :** 9 entrypoints, 22 codes d'erreur, 11 tests automatisés validés
- **Résistance Sybil prouvée :** Le vote pondéré par réputation neutralise les comptes fantômes
- **Double mode de finalisation :** Garantit la résolution même en cas de faible participation
- **Pattern pull-over-push :** Suppression des boucles, `claim_reward` individuel, sécurité gas garantie
- **Anti-spam économique :** Le dépôt de soumission pénalise les fausses in-

formations
- **Verrouillage de stake :** `active_votes` empêche tout retrait pendant un vote actif
- **Interface complète :** Mode Démo intégré pour tester sans déploiement réel
- **Gouvernance paramétrable :** L'admin peut ajuster 6 paramètres sans redéploiement

9.1 Perspectives d'amélioration

- **Pénalité de stake :** Déduire une partie du stake des mauvais voteurs
- **Catégorisation :** Organiser les articles par thématiques (politique, science, sport)
- **Mécanisme d'appel :** Permettre le recours après finalisation
- **DAO Governance :** Remplacer l'admin unique par un vote collectif des membres confirmés
- **Oracle externe :** Intégrer des sources tierces de fact-checking
- **Mainnet :** Migration vers le réseau principal Tezos

NewsChecker DApp

Smart Contract SmartPy • Tezos Ghostnet • React 19 • Taquito 20 • Beacon SDK 4

https://github.com/votre-repo/news-checker-dapp